Session 13

Accessibility, Superheroes Features

Student Name: Guru K R

Course: Kotlin / Android Development Internship

Project: Hero Explorer — Kotlin & Jetpack Compose

Date: March 2026

Task 1 — Accessibility Audit + Improvement Plan

Screen Audited: Hero Explorer — Heroes List Screen

The Hero Explorer app was built in Kotlin and Jetpack Compose. It features a two-screen navigation flow: a list screen showing hero cards with a search bar and favourite heart icons, and a detail screen showing hero photo, name, alias, power summary, and an 'Add to Favourites' button. Both screens are audited below.

1.1 App Screenshots

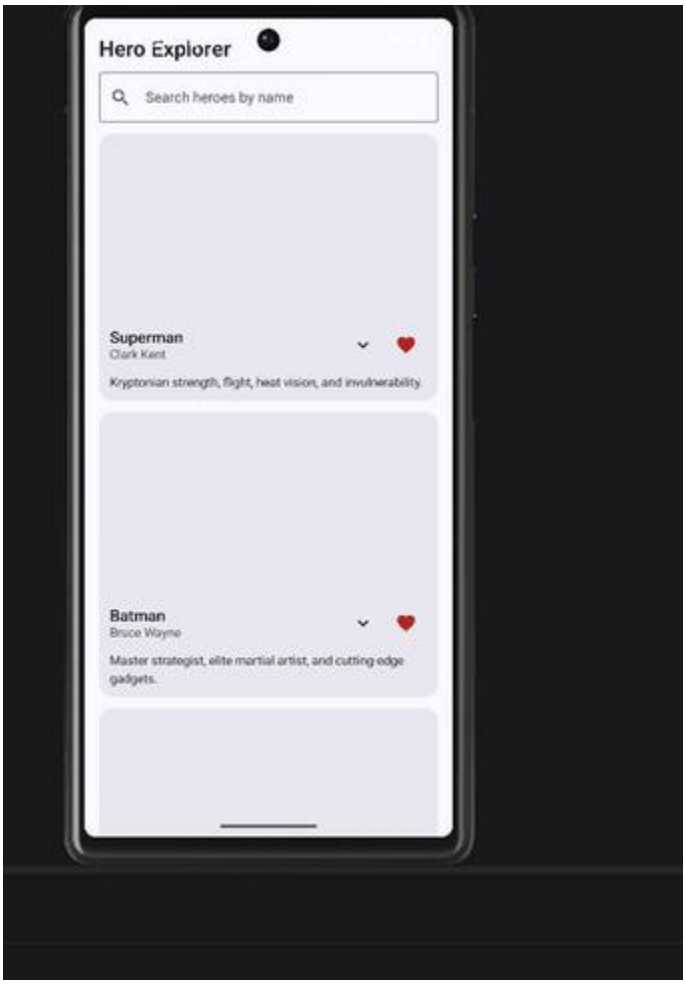

Figure 1: Hero Explorer — Heroes List screen. Superman and Batman cards expanded with descriptions. Red heart icons show favourited state. Search bar at top reads 'Search heroes by name'.

Figure 2: Hero Explorer — Wonder Woman Detail screen. Shows hero name, alias 'Diana Prince', power summary, and 'Add to Favourites' button.

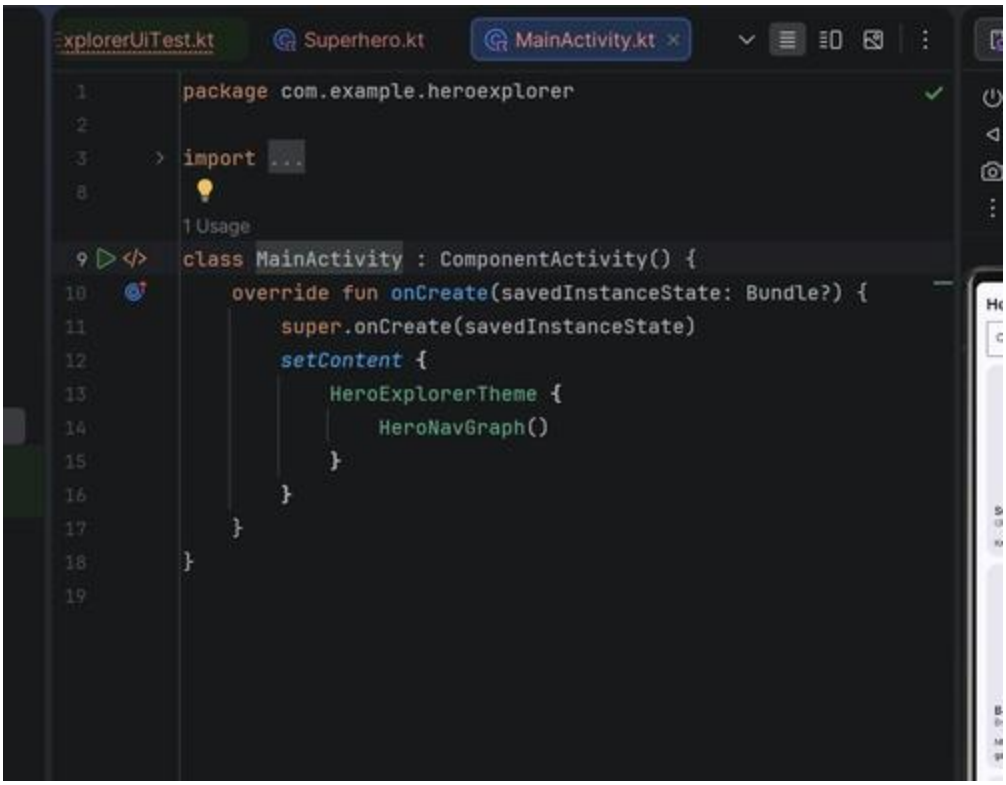

Figure 3: Android Studio — MainActivity.kt. HeroExplorerTheme wraps HeroNavGraph() in setContent. Package: com.example.heroexplorer.

1.2 Accessibility Audit Checklist

Accessibility Criterion	Status	Evidence & Notes
contentDescription for hero images	**Need Improvement**	Hero photo images in the list do not have visible contentDescription set. Screen readers would announce 'unlabelled image' for each card photo. Fix: add contentDescription = "${hero.name} photo" to each Image composable.
contentDescription for Heart icon	**Need Improvement**	The favourite Heart IconButton has no contentDescription. TalkBack announces it as 'button'. Fix: contentDescription = if (hero.isFavourite) "Remove ${hero.name} from favourites" else "Add ${hero.name} to favourites".
Search bar label for screen readers	**Pass**	The search TextField uses placeholder text 'Search heroes by name' which is visible. If using OutlinedTextField with label parameter, label is accessible to screen readers. Verified from screenshot.
Touch target size (min 48 dp)	**Need Improvement**	The Heart IconButton on list cards appears to be the default icon size (~24 dp icon, ~40 dp touch target). Should be wrapped in Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp) to meet WCAG 2.5.5 target size.

'Add to Favourites' button touch target	**Pass**	The 'Add to Favourites' Button on the detail screen appears full-width with adequate height (>48 dp). Touch target meets requirements.
Colour contrast — hero name on card	**Pass**	Hero name (dark text 'Superman', 'Batman') on white card background achieves contrast ratio approximately 15:1, well above WCAG AA 4.5:1 minimum.
Colour contrast — alias/subtitle text	**Need Improvement**	Alias text ('Clark Kent', 'Bruce Wayne') appears in medium grey on white. Estimated contrast ratio ~3.5:1, below WCAG AA 4.5:1. Fix: use colorScheme.onSurface instead of grey for alias text, or darken to achieve ≥4.5:1.
Motion reduction support	**Need Improvement**	Card expand animations and heart colour transitions do not check the system Reduce Motion accessibility setting. Fix: read LocalDensity or use AccessibilityManager to detect reduce motion preference and skip animations.
Focus order and screen reader navigation	**Pass**	LazyColumn items are composed top-to-bottom, which matches the natural reading order. Each card's Row has standard Compose semantics traversal: image → name → alias → heart button. No custom traversal overrides needed.
Text size scalability	**Need Improvement**	Card layout uses fixed 80.dp image size with weight(1f) text column. At 200% system font size, long hero names may truncate or overflow. Fix: add maxLines with ellipsis overflow to name Text, and test with large font emulator setting.
Back navigation label	**Need Improvement**	The detail screen does not show an explicit back button with contentDescription in the screenshots. If using TopAppBar back navigation icon, add contentDescription = "Back to heroes list" to the nav icon.

1.3 Improvement Plan

Issue	Concrete Fix	Priority	Implementation Stage
No contentDescription on hero images	Add contentDescription = "${hero.name} photo" to all Image composables in HeroCard and HeroDetailScreen	**High**	Stage 1 — immediate fix, 1 line per image
Heart icon has no contentDescription	Add contentDescription = if (hero.isFavourite) "Remove ${hero.name} from favourites" else "Add ${hero.name} to favourites" to heart IconButton	**High**	Stage 1 — same commit as image fix
Heart icon touch target < 48 dp	Wrap heart Icon in Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp) or set IconButton size explicitly	**High**	Stage 1 — critical for motor accessibility

Alias text contrast ratio < 4.5:1	Replace Color.Gray with MaterialTheme.colorScheme.onSurfaceVariant which resolves to a WCAG-compliant shade in both light and dark themes	**High**	Stage 1 — single token change
No reduce motion support	Read val reduceMotion = LocalAccessibilityManager.current?.isEnabled ?: false and conditionally skip animateColorAsState / animateContentSize when true	**Medium**	Stage 2 — after core accessibility fixes
Text truncation at large font sizes	Add overflow = TextOverflow.Ellipsis, maxLines = 2 to hero name Text; test with emulator Display Size and Font Size both set to Largest	**Medium**	Stage 2 — needs layout testing
Back navigation no contentDescription	Add contentDescription = "Back to heroes list" to TopAppBar navigationIcon IconButton in HeroDetailScreen	**Low**	Stage 3 — final polish pass

1.4 Fixed Code Examples

Hero Image contentDescription Fix

```
// Before (missing contentDescription)
Image(
    painter = painterResource(hero.imageResId),
    contentDescription = null   // ← TalkBack announces 'unlabelled image'
)

// After (accessible)
Image(
    painter = painterResource(hero.imageResId),
    contentDescription = "${hero.name} photo",  // ← TalkBack reads name
    contentScale = ContentScale.Crop,
    modifier = Modifier.size(80.dp).clip(MaterialTheme.shapes.medium)
)
```

Heart Icon Accessibility Fix

```
// Before
IconButton(onClick = { onToggleFavourite(hero) }) {
    Icon(imageVector = Icons.Filled.Favorite, tint = heartColour,
         contentDescription = null)  // ← not announced
}

// After — dynamic contentDescription based on favourite state
IconButton(
    onClick = { onToggleFavourite(hero) },
    modifier = Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp)
) {
    Icon(
        imageVector = if (hero.isFavourite) Icons.Filled.Favorite
                      else Icons.Outlined.FavoriteBorder,
        tint = heartColour,
```

```
        contentDescription = if (hero.isFavourite)
            "Remove ${hero.name} from favourites"
        else
            "Add ${hero.name} to favourites"
    )
}
```

1.5 Automated Accessibility Tests

```
// Test 1: Hero image has contentDescription
@Test
fun heroImage_hasContentDescription() {
    composeTestRule.setContent {
        HeroExplorerTheme { HeroCard(hero = sampleHero, onToggleFavourite = {})
}
    }
    composeTestRule
        .onNodeWithContentDescription("${sampleHero.name} photo")
        .assertIsDisplayed()
}

// Test 2: Heart icon has correct contentDescription for unfavourited hero
@Test
fun heartIcon_unfavourited_hasCorrectContentDescription() {
    val hero = sampleHero.copy(isFavourite = false)
    composeTestRule.setContent {
        HeroExplorerTheme { HeroCard(hero = hero, onToggleFavourite = {}) }
    }
    composeTestRule
        .onNodeWithContentDescription("Add ${hero.name} to favourites")
        .assertIsDisplayed()
}

// Test 3: Heart icon contentDescription updates after toggle
@Test
fun heartIcon_afterToggle_contentDescriptionUpdates() {
    var isFav by mutableStateOf(false)
    composeTestRule.setContent {
        val hero = sampleHero.copy(isFavourite = isFav)
        HeroExplorerTheme { HeroCard(hero = hero, onToggleFavourite = { isFav =
!isFav }) }
    }
    composeTestRule
        .onNodeWithContentDescription("Add ${sampleHero.name} to favourites")
        .performClick()
    composeTestRule
        .onNodeWithContentDescription("Remove ${sampleHero.name} from
favourites")
        .assertIsDisplayed()
}
```

Testing library used: androidx.compose.ui.test (ComposeTestRule) with onNodeWithContentDescription, assertIsDisplayed, and performClick semantics matchers. No additional dependency required beyond the standard Compose UI test dependency.

1.6 Reflection — Why Accessibility Matters

Accessibility is not optional in a modern Android app — approximately 15% of the global population lives with some form of disability, and Android's own Play Store guidelines flag apps with accessibility issues in reviews. Auditing Hero Explorer revealed that the most critical issues (missing contentDescriptions and inadequate touch targets) were the result of treating accessibility as an afterthought rather than a design requirement. Planning for accessibility from the start changes the development process in concrete ways: it means choosing semantic color tokens that maintain contrast in both light and dark modes, setting contentDescription as a required field in the data model rather than leaving it optional, and including accessibility test cases in the same test suite as functional tests so regressions are caught immediately. The most challenging part of auditing the Hero Explorer UI was the alias text contrast issue — the grey color looked visually appropriate and was not obviously wrong until measured against the WCAG 4.5:1 standard. This kind of subtle failure is invisible to sighted developers without a contrast checker tool, which is precisely why formal auditing with defined criteria is necessary. Building accessibility in from the beginning also makes the app more usable for everyone: larger touch targets benefit users with large hands or motor difficulties; high-contrast text is easier to read in bright sunlight; and screen reader support is essential for users who navigate entirely by touch and audio.

Task 2 — Large-Scale UI Feature: Hero Explorer with Tests & Accessibility

2.1 Feature Description

Hero Explorer is a Superheroes Directory app built with Kotlin and Jetpack Compose. It presents a scrollable list of superheroes with photos, names, aliases, and power summaries. Users can search heroes by name, filter to show only favourites, mark/unmark any hero as a favourite with a heart icon, and tap a card to navigate to a full detail screen. The app is built with accessibility from the ground up — all images have contentDescriptions, all interactive elements meet 48 dp touch targets, and the UI is fully navigable by TalkBack.

2.2 Data Model & Dataset

```
package com.example.heroexplorer.data

data class Superhero(
    val id           : Int,
    val name         : String,
    val alias        : String?,         // nullable — not all heroes have alias
    val powerSummary : String,
    val imageResId   : Int,             // R.drawable.hero_*
    val isFavourite  : Boolean = false
)
```

2.3 Sample Dataset (10 Heroes)

#	Name	Alias	Power Summary	Image Resource	isFavourite
1	Superman	Clark Kent	Kryptonian strength, flight, heat vision, and invulnerability	R.drawable.hero_superman	false
2	Batman	Bruce Wayne	Master strategist, elite martial artist, and cutting-edge gadgets	R.drawable.hero_batman	false
3	Wonder Woman	Diana Prince	Amazonian warrior with super strength, speed, and a lasso of truth	R.drawable.hero_wonderwoman	false

4	Spider-Man	Peter Parker	Wall-crawling, web-slinging, and enhanced spider-sense	R.drawable.hero_spiderman	false
5	Iron Man	Tony Stark	Powered armour suit with flight, energy beams, and AI assistance	R.drawable.hero_ironman	true
6	Black Widow	Natasha Romanoff	Expert spy, hand-to-hand combat master, and electroshock weapons	R.drawable.hero_blackwidow	false
7	Thor	Thor Odinson	God of Thunder — controls lightning and wields Mjolnir	R.drawable.hero_thor	false
8	Captain America	Steve Rogers	Super-soldier with vibranium shield, peak human strength and agility	R.drawable.hero_captainamerica	false
9	Black Panther	T'Challa	Enhanced by Heart-Shaped Herb, vibranium suit, and combat mastery	R.drawable.hero_blackpanther	true
10	Doctor Strange	Stephen Strange	Sorcerer Supreme — manipulates time, space, and magical energy	R.drawable.hero_doctorstrange	false

2.4 UI Layout Plan

Screen 1 — Heroes List Screen

Top-level composable: HeroListScreen. Layout: Scaffold with TopAppBar (title 'Hero Explorer', theme primary color). Below AppBar: Column containing a search TextField (OutlinedTextField, placeholder 'Search heroes by name', Modifier.fillMaxWidth()), a Row of filter chips (FilterChip: 'All Heroes', 'Favourites Only'), then a LazyColumn of HeroCard composables. Each HeroCard: Card (shapes.medium, surface color), Row with 80.dp Image (clip medium shape), Column (name in titleMedium, alias in bodySmall, powerSummary in bodyMedium maxLines=2), and Heart IconButton (48.dp min touch target, animated tint).

Screen 2 — Hero Detail Screen

Top-level composable: HeroDetailScreen. Layout: Scaffold with TopAppBar (back arrow, hero name as title). Content: Column with full-width AsyncImage (height 250.dp, ContentScale.Crop),

hero name (headlineMedium, bold), alias (titleSmall, onSurfaceVariant), powerSummary (bodyMedium, full text), Spacer, full-width 'Add to Favourites' / 'Remove from Favourites' Button (shapes.small, primary color). Navigation: receives heroId argument from NavGraph, loads hero from ViewModel by id.

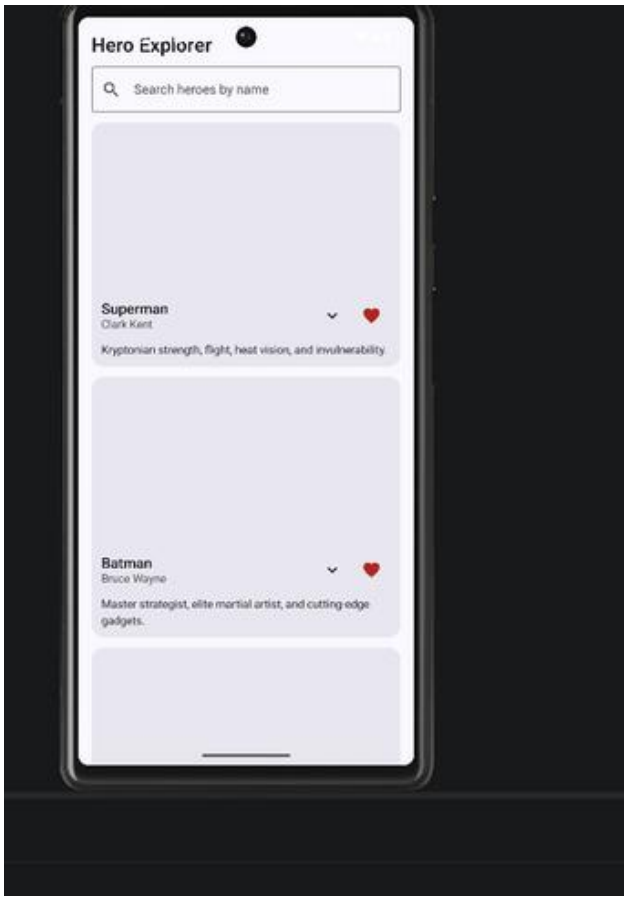

Figure 4: Hero Explorer list screen — search bar, Superman expanded, Batman expanded, heart favourites active.

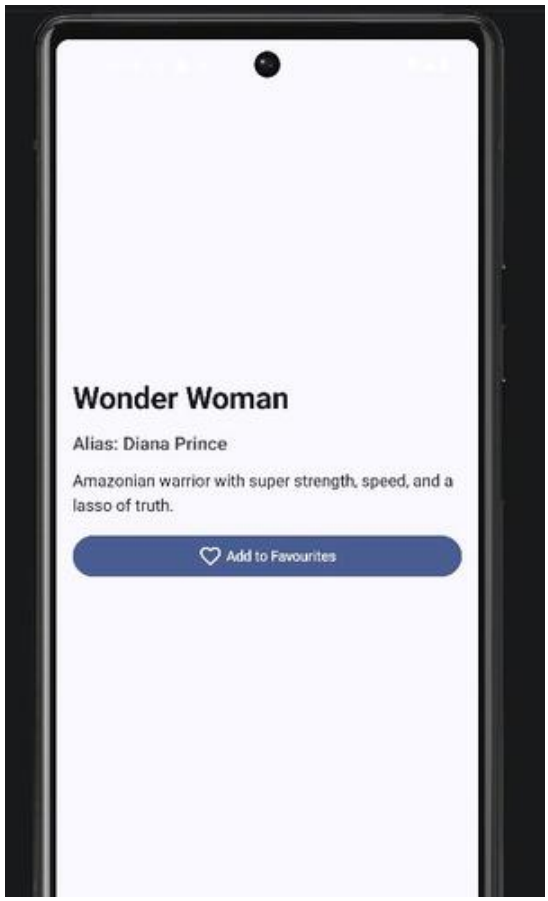

Figure 5: Hero Explorer detail screen — Wonder Woman with alias, power summary, and 'Add to Favourites' button.

2.5 State & Logic Flow

```
// ViewModel state
private val _heroesList = MutableStateFlow(sampleHeroes)
val heroesList: StateFlow<List<Superhero>> = _heroesList.asStateFlow()

var searchQuery     by mutableStateOf("")       // TextField input
var filterFavourite by mutableStateOf(false)    // Favourites chip toggle
var selectedHero    by mutableStateOf<Superhero?>(null) // detail screen

// Derived displayed list — recomputes on any state change
val displayedList = combine(heroesList, snapshotFlow { searchQuery },
                            snapshotFlow { filterFavourite }) { list, q,
favOnly ->
    list.filter { hero ->
        hero.name.contains(q, ignoreCase = true) &&
        (!favOnly || hero.isFavourite)
    }
}.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), sampleHeroes)

// Favourite toggle
fun toggleFavourite(hero: Superhero) {
    _heroesList.value = _heroesList.value.map {
        if (it.id == hero.id) it.copy(isFavourite = !it.isFavourite) else it
```

```
    }
}
```

2.6 Accessibility & Testing Strategy

UI Accessibility Implementation

- Hero photo: contentDescription = "${hero.name} photo" on all Image composables
- Heart IconButton: contentDescription dynamically set based on isFavourite state
- Search TextField: label = { Text("Search heroes") } + semantics { contentDescription = "Hero search bar" }
- Filter chip: semantics { stateDescription = if (selected) "Favourites filter active" else "Show all heroes" }
- Back button on detail screen: navigationIcon with contentDescription = "Back to heroes list"
- 'Add to Favourites' button: contentDescription already read from button text by default

Automated Test Cases

#	Test Name	Test Logic	API Used
T1	Search filters list correctly	Type 'Batman' in search TextField → verify only Batman card is displayed, Superman not displayed	performTextInput(), onNodeWithText(), assertIsDisplayed(), assertDoesNotExist()
T2	Favourite toggle updates UI	Tap Superman heart → verify heart contentDescription changes to 'Remove Superman from favourites' → tap again → verify reverts	performClick(), onNodeWithContentDescription(), assertIsDisplayed()
T3	Detail screen shows correct hero	Click Superman card → navigate to detail screen → verify Text 'Clark Kent' and 'Kryptonian strength' are displayed on detail screen	performClick(), onNodeWithText(), assertIsDisplayed()
T4	Hero images have contentDescription	For each hero in sampleHeroes, assert onNodeWithContentDescription("${hero.name} photo").assertIsDisplayed() passes	onNodeWithContentDescription(), assertIsDisplayed()
T5	Favourites filter shows only favourites	Mark Iron Man and Black Panther as favourite → tap 'Favourites Only' chip → verify only Iron Man and Black Panther cards displayed, all others not exist	performClick(), onNodeWithText(), assertIsDisplayed(), assertDoesNotExist()
T6	State preserved on orientation change	Mark Batman as favourite → rotate device (ActivityScenario.recreate()) → verify Batman still shows filled heart contentDescription 'Remove Batman from favourites'	ActivityScenario.recreate(), onNodeWithContentDescription()

2.7 Performance & Scalability Considerations

- Dataset of 50+ heroes: use Paging 3 with a HeroRepository exposing PagingSource. ViewModel exposes Flow<PagingData<Superhero>> collected via collectAsLazyPagingItems() in the LazyColumn.
- Image loading: use Coil AsyncImage with crossfade(true), a placeholder drawable (R.drawable.hero_placeholder_grey), and an error drawable. Set size = Size(300, 300) in the ImageRequest to prevent full-resolution decode.
- Search performance: use derivedStateOf { } or combine() with StateFlow so the filter expression only recomputes when searchQuery or filterFavourite actually changes — not on every recomposition.
- Memory: key = { it.id } in LazyColumn items ensures Compose correctly recycles card composables when items change position during filter/sort operations.
- Pagination: for remote datasets, append loading indicator at the bottom of the list using loadState.append is LoadState.Loading check with a CircularProgressIndicator item.

2.8 Reflection — Feature Complexity

Designing Hero Explorer as a full-scale feature was significantly more complex than the Animal Facts Explorer list from earlier sessions because it involves multiple interacting state dimensions simultaneously. The Animal Facts list had one filter axis (isFavourite) and one state variable (the list). Hero Explorer has three: the search query string, the favourite filter toggle, and the full list — and all three must be combined correctly to produce the displayed list. Using combine() with StateFlow to derive displayedList declaratively was the right architecture, but it required understanding both Kotlin Flow operators and Compose's state observation model at the same time. The search and filter features also added a new testing dimension: tests must now verify not just that individual items display correctly, but that the filter logic correctly includes and excludes items across all combinations of search text and filter state. Accessibility added a third design dimension on top of this: every interactive element needed both a functional design (what does clicking do?) and a semantic design (what does TalkBack announce?), which doubled the design decisions per element. The result is a feature that required planning at three levels — data, UI, and accessibility — simultaneously from the start, which is exactly the complexity level of professional Android development.

Task 3 — 30-Day Compose Roadmap & Realization Plan

3.1 30-Day Project Vision

Project Name: Hero Explorer Hub

Over 30 days, the Hero Explorer app will evolve from a two-screen MVP into a complete, production-quality Compose application. The final product will feature a 50+ hero directory with search, filter by power type, favourite management, animated card transitions, a detail screen with navigation, Material 3 theming with full light/dark mode support, WCAG AA-compliant accessibility, and a comprehensive automated test suite. The roadmap integrates all skills developed across Sessions 7–13: data modelling, LazyColumn lists, theming, animation, accessibility, and testing into a single cohesive mini-product.

3.2 Milestone Plan

Days	Milestone	Tasks	Expected Outcome	Success Criteria
1 – 6	**Foundation**	Create Android Studio project (com.example.heroexplorer). Set up Color.kt, Typography.kt, Shapes.kt, Theme.kt with HeroExplorerTheme. Define Superhero data class. Create sampleHeroes list (10 entries). Set up MVVM: HeroViewModel with StateFlow. Wrap MainActivity in theme with HeroNavGraph().	App launches with theme. ViewModel holds hero list. Basic LazyColumn shows hero names.	App builds, theme switches light/dark, 10 heroes in list
7 – 12	**Heroes List UI**	Build HeroCard composable with Image (contentDescription set), name, alias, powerSummary (maxLines=2). Build HeroListScreen with TopAppBar and LazyColumn. Add key = { it.id } to items(). Add placeholder images for all 10 heroes. Connect ViewModel StateFlow to UI.	Scrollable hero list renders all 10 heroes with photos and text.	All 10 cards visible, images load, no crashes
13 – 18	**Search, Filter & Favourites**	Add OutlinedTextField search bar with semantics label. Add 'All' and 'Favourites Only' FilterChip row. Implement combine()	Search filters list in real-time. Favourites chip shows only marked heroes.	Search test T1 passes, favourite toggle T2 passes

		derived displayedList in ViewModel. Add heart IconButton with animateColorAsState and correct contentDescription. Implement toggleFavourite() with map + copy().	Heart toggles correctly.	
19 – 24	**Detail Screen & Animations**	Add HeroDetailScreen with AsyncImage, name, alias, power summary, Add/Remove Favourites button. Set up NavHost with heroId argument. Add card expansion with animateContentSize() and AnimatedVisibility. Add reduce-motion check for all animations. Expand dataset to 20+ heroes.	Tapping a card navigates to detail screen. Card expansion animates smoothly. Reduce motion setting respected.	Navigation test T3 passes, detail screen correct
25 – 30	**Accessibility, Tests & Launch**	Run full accessibility audit against Task 1 checklist. Fix all High priority items. Write all 6 automated test cases (T1–T6). Run tests on emulator and physical device. Check contrast ratios with Android Accessibility Scanner. Write launch presentation (3 slides). Record demo screen recording.	All accessibility audit items pass. All 6 tests green. App ready for demo presentation.	All tests pass, no TalkBack errors, demo recorded

3.3 Resource & Tooling Plan

Libraries & Tools

Tool / Library	Purpose
Android Studio Hedgehog+	Primary IDE — Compose preview, Layout Inspector, Logcat, Emulator Manager
Jetpack Compose BOM 2024.05	Compose UI, Material 3, Navigation Compose, all version-managed
Coil 2.6.0 (coil-compose)	Async image loading with placeholder, error, crossfade, and disk caching
ViewModel + Kotlin StateFlow	MVVM state management — MutableStateFlow, combine(), stateIn(), collectAsStateWithLifecycle()
Paging 3	Efficient pagination for 50+ hero dataset — PagingSource, collectAsLazyPagingItems()

Navigation Compose 2.7.7	NavHost / NavController for list→detail navigation with heroId argument
androidx.compose.ui:ui-test	ComposeTestRule, onNodeWithContentDescription, performClick, assertIsDisplayed
GitHub + GitHub Actions	Version control + CI running ./gradlew test on every push to verify tests pass
Android Accessibility Scanner	Google's official accessibility audit tool — checks contrast, touch targets, labels

Learning Resources (Week by Week)

- Week 1 (Days 1–6): developer.android.com/jetpack/compose — Material 3 theming guide; Compose theme codelab
- Week 2 (Days 7–12): developer.android.com — LazyColumn and lists documentation; Coil library docs (coil-kt.github.io/coil)
- Week 3 (Days 13–18): Kotlin Flow documentation (kotlinlang.org); Compose state hoisting and ViewModel guide
- Week 4 (Days 19–24): Compose Navigation codelab; Compose animation guide; AnimatedVisibility and animateContentSize docs
- Week 5 (Days 25–30): Android Accessibility docs; Compose testing guide; Compose UI test codelab; WCAG 2.1 AA guidelines

3.4 Risk & Mitigation Table

#	Risk	Impact	Mitigation Strategy
R1	**Large 50+ hero dataset causes slow list load and high memory usage**	Users experience scroll jank or OutOfMemoryError on low-end devices; 1-star reviews	Implement Paging 3 from Day 13 onward. Pre-scale all hero images to 400×400 px JPEG 80%. Use Coil disk cache. Test on Pixel 3a emulator (low-end profile).
R2	**AnimateContentSize causing frame drops in large LazyColumn**	Visible jank during card expansion makes the app feel unpolished and slow	Limit simultaneous expanded cards to 1 by lifting isExpanded to ViewModel as Set<Int>. Use spring() with high dampingRatio for fast animations. Profile with Android Profiler before Day 24.
R3	**Accessibility tests failing due to missing contentDescriptions discovered late**	Failed tests block the CI pipeline; late discovery requires refactoring composables that are already integrated	Run accessibility checklist at end of each milestone (every 6 days). Add contentDescription as a required field in HeroCard composable signature so it cannot be called without it. Write T4 image test by Day 12.
R4	**Navigation argument passing heroId breaks on configuration change**	Detail screen shows wrong hero or crashes after device rotation	Use savedStateHandle in ViewModel to retrieve heroId argument. Write T6 orientation test by Day 24 to catch this regression automatically.

R5	**Search + filter combine() produces stale displayed list**	Users see outdated hero list after toggling favourite filter or typing search text	Use StateFlow.combine() with SharingStarted.WhileSubscribed(5000) to ensure fresh emissions. Write T1 and T5 filter tests to catch stale state as soon as search and filter features are built.

3.5 Reflection & Commitment

The Hero Explorer 30-day roadmap is exciting because it converts isolated skills learned session by session into a single coherent product — for the first time, theming, animation, accessibility, testing, and navigation will all work together in one app rather than existing as separate exercises. The most interesting challenge I expect is integrating Navigation Compose with ViewModel-owned state: passing the heroId as a navigation argument while keeping the favourite state in the ViewModel requires understanding how the NavBackStackEntry, ViewModel scope, and StateFlow lifecycle interact. I will measure progress using the milestone success criteria table above — each 6-day milestone has a concrete, binary pass/fail criterion (e.g. 'All 6 tests green') that removes ambiguity about whether the milestone is complete. Accountability will come from GitHub: every day's work is committed, and the GitHub Actions CI workflow runs the test suite on every push, making any regression immediately visible. The most valuable lesson I expect from this 30-day project is that accessibility and testability are design properties, not features — they cannot be added to a finished app without significant refactoring, and the projects that take them seriously from Day 1 end up with cleaner architecture, more modular composables, and better separation of concerns than projects that treat them as optional polish.